DAT505: Assignment 2

Marius Langeland

November 2026

1 Introduction

Computers and servers are not the only attack surfaces available to an attacker: in principle any device connected to a system can be used against it. In this assignment the surface is printers, specifically the Common UNIX Printing System (CUPS) and its `cups-browsed` component. The lab environment is labelled CVE-2024-47177, which addressed the remote-code-execution aspect of this attack surface. That identifier has since been marked *Rejected* by the publishing CNA as a duplicate of the related CUPS vulnerabilities CVE-2024-47076, CVE-2024-47175, and CVE-2024-47176, which together form the actual exploit chain.

The goal of this assignment is to show how easily an attacker can advertise a printer under their own control and make CUPS connect back to it. This work deliberately covers only the *discovery* step, the malicious advertisement and the victim's callback, and not the full remote-code-execution chain. I also cover how such CUPS attacks can be detected and mitigated, and run a series of experiments in an isolated environment under my full control using Vulhub.

2 Environment and Tool Implementation

2.1 Lab environment

All experiments were performed in an isolated Docker environment based on `vulhub/cups-browsed/CVE-2024-47177`. No external, university, or third-party system was involved. The roles were:

- **Victim:** the vulnerable `cups-browsed` container (version 2.0.1), address `172.18.0.2`.
- **Attacker:** the Docker host, `172.18.0.1`, running `cups_attack.py`.
- **Detector:** `cups_detector.py` sniffing the bridge interface `br-60480b7c1bbb`.
- **Printer:** a software `ippeveprinter` on the host (`ippeveprinter -p 8632 "Test Printer"`), with no physical printer required.

2.2 Attack tool (`cups_attack.py`)

The attack tool was developed independently, as required. Its purpose is primarily to send a discovery announcement to the vulnerable CUPS system. It has a listener attached that detects whether the CUPS system tries to connect back to the attacking printer. It does not implement remote code execution.

If the `--no-listen` flag is not set, the program first starts the listener. It then builds a UDP datagram and sends it to the target, using the configured target port and source port. The payload consists of six fields:

- Type: the kind of printer, in hex.
- State: an IPP enum, where 3 = idle, 4 = processing, 5 = stopped.
- URI: the address of the printer.
- Location: cosmetic.
- Info: free-text description.
- Make and model: the printer make-and-model string.

This is the legacy CUPS browse format, which remains in place to let legacy printers interoperate with newer systems. The tool can set all six fields freely, but the flags are aimed at changing the URI, which is where the CUPS system is tricked into connecting to a malicious "printer". An example payload:

```
0x1000002 3 http://172.18.0.1:9999/printers/labtest1 "lab" "Evil Printer" "Generic PostScript"
```

The two most important flags are `--target` and `--adv-ip`. The program also sets defaults, such as a broadcast target and standard ports for both the target and the attacker.

- `--target`: the target IP.
- `--target-port`: the target port.
- `--adv-ip` (required): the malicious IP.
- `--adv-port`: the malicious port.
- `--iface`: the interface to sniff on.
- `--listen-timeout`: how long the sniffer listens before shutting down.
- `--no-listen`: disables the sniffer.
- `--queue`: the printer endpoint.

For experimentation and proof, a few extra flags are available:

- `-v`, `--verbose`: the logger outputs debug lines.

- `--pcap`: the sniffer captures the packets to a file.
- `--state`: defaults to 3, and can be changed to make the packet malformed for the detector tool.
- `--printer-type`: a non-hex value makes the packet malformed.

2.3 Detection tool (`cups_detector.py`)

The detection tool passively monitors the network for signs of a CUPS discovery attack. It sniffs with Scapy on a chosen interface and applies the BPF filter `udp port 631 or tcp`, which captures both legacy browse announcements and the IPP/TCP connections that may follow. Capture happens at the device layer, which means packets are seen before any host firewall `INPUT` filtering.

Each announcement is parsed by `parse_browse()` into the six browse fields. Packets that do not split into exactly six fields are ignored. The tool then raises one of four alerts:

- `[UNAUTHORIZED AD]`: the advertisement came from a source that is not in the allowlist.
- `[SUSPICIOUS IPP]`: a TCP connection was made to a printer that a non-allowlisted source had advertised.
- `[MALFORMED]`: a field failed validation. The checks are a valid hex `type`, a `state` of 3, 4 or 5, a URI scheme of http, ipp or https, a known URI host, and a URI port of 631.
- `[FLOOD]`: one source sent more announcements than the threshold within the time window.

The behaviour is controlled by the following flags:

- `--iface`: the interface to sniff on.
- `--filter`: the BPF capture filter (default `udp port 631 or tcp`).
- `--known-printers`: path to a CSV allowlist of authorised printers (`ip,name,port`).
- `--window`: the flood time window in seconds (default 10).
- `--threshold`: the number of announcements within the window before a flood is reported (default 5).
- `--csv`: write the alerts to a CSV file.
- `--json`: write the alerts to a JSON file.
- `-v`, `--verbose`: the logger outputs debug lines.

Every alert is printed as a log line and, when `--csv` or `--json` is given, also written to that file with the columns `timestamp`, `alert`, `source`, and `detail`. CSV rows are written as each alert fires, and the JSON file is written when the detector stops.

3 Attack Simulation and Analysis

Two allowlist configurations were used throughout: an *empty* list (every advertiser unknown) and a *populated* list containing `OfficeLaser` (`172.18.0.50`), `ReceptionMFP` (`172.18.0.51`) and `LabPrinter` (`172.18.0.1:631`).

3.1 Experiment 1: Normal traffic

A well-formed advertisement for `LabPrinter` on the standard IPP port `631`, from an allowlisted address, produced *no* detector warnings. The victim created the queue and a full bidirectional IPP session followed (capture `exp1_normal.pcap`, 51 packets):

```
[INFO] Known printers: {... '172.18.0.1': {'name':'LabPrinter','port':'631'}}
[INFO] Parsed advertisement: {... 'uri':'http://172.18.0.1:631/printers/LabPrinter' ...}
   (no warnings)
[INFO] Captured: IP / TCP 172.18.0.2:48708 > 172.18.0.1:ipp S   ... (full IPP exchange)
```

This is the true-negative baseline: well-formed, allowlisted, standard-port traffic is correctly left unflagged. (Note: this traffic is produced by the attack tool advertising an allowlisted printer, which makes it a *legitimate-looking* baseline rather than an organic print job. It is the closest available benign reference.)

Evidence. `evidence/detector_exp1_allowlisted.log` (the advertisement is parsed at line 4 with no following alert) and `pcap_files/exp1_normal.pcap`. The same advertisement with an empty allowlist is instead flagged at line 5 of `evidence/detector_exp1_noallowlist.log`.

3.2 Experiment 2: Unauthorized discovery

An advertisement sent with an *empty* allowlist is correctly identified as hostile, and the coerced callback is flagged (`detector_before.log`):

```
[WARNING] [UNAUTHORIZED AD] 172.18.0.1 advertised 172.18.0.1:8631
[INFO] Captured: IP / TCP 172.18.0.2:37952 > 172.18.0.1:8631 S
[WARNING] [SUSPICIOUS IPP] connection to attacker-advertised printer 172.18.0.1:8631
```

The attacker tool independently confirms the victim connecting back (`>>> VICTIM CONNECTED BACK`, HTTP `POST` received). The victim retried the callback several times, each producing a fresh `[SUSPICIOUS IPP]` alert.

Evidence. `evidence/detector_before.log` (the `[UNAUTHORIZED AD]` at line 5 and the first `[SUSPICIOUS IPP]` at line 8) and `pcap_files/before_mitigation.pcap`. The detector output is shown in Figure 1.

3.3 Experiment 3: Packet manipulation

Four crafted variations were sent, each targeting a different field checked by `validate()`. The advertiser's address was allowlisted in these runs, which (deliberately) exposes the detection gap discussed in Section 3.4.
Representative detector lines:

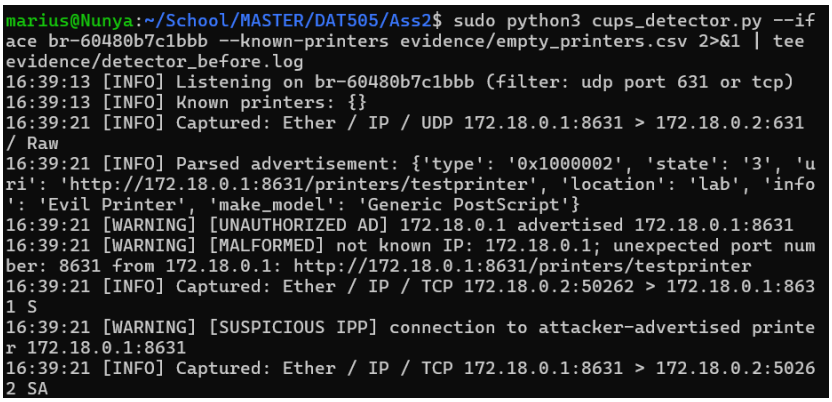


Figure 1: Detector output for the unauthorized-discovery run. The advertisement raises `[UNAUTHORIZED AD]` and `[MALFORMED]`, and the victim's callback raises `[SUSPICIOUS IPP]`.

Var.	Crafted field	Detector alert(s)	Victim callback
v1	`printer-state=99`	`[MALFORMED]` state + port	SYN, then reset
v2	`type=zzz`	`[MALFORMED]` type + port	none
v3	`port=9999`	`[MALFORMED]` port	full session
v4	flood (8×)	`[FLOOD]` + `[MALFORMED]`	SYN per advert

Table 1: Crafted discovery messages and the detector's response.

```
v1: [MALFORMED] abnormal printer-state: 99; unexpected port number: 9999
v2: [MALFORMED] type not valid hex: zzz; unexpected port number: 9999
v3: [MALFORMED] unexpected port number: 9999
v4: [FLOOD] 172.18.0.1 sent 6 announcements in 10s   (repeats at 7, 8)
```

The flood alert fires on the sixth announcement, consistent with the default `--threshold 5` over `--window 10` (Figure 2). The `cups-browsed` daemon log corroborates the behaviour: it accepts each browse packet, attempts `get-printer-attributes` against `ipp://172.18.0.1:9999/...`, fails (the fake printer returns nothing), and logs `incorrect browse packet format` for the unparsable variants.

Evidence. Each variation has a matching detector log and capture:

- v1: `evidence/detector_exp3_v1_state.log` (alert at line 5), `pcap_files/exp3_v1_state.pcap`
- v2: `evidence/detector_exp3_v2_type.log` (alert at line 5), `pcap_files/exp3_v2_type.pcap`
- v3: `evidence/detector_exp3_v3_port.log` (alert at line 5), `pcap_files/exp3_v3_port.pcap`
- v4: `evidence/detector_exp3_v4_flood.log` (the `[FLOOD]` alerts, first on the sixth announcement), `pcap_files/exp3_v4_flood.pcap`

The daemon-side behaviour is recorded in `evidence/cupsbrowsed_exp3_all.log`.

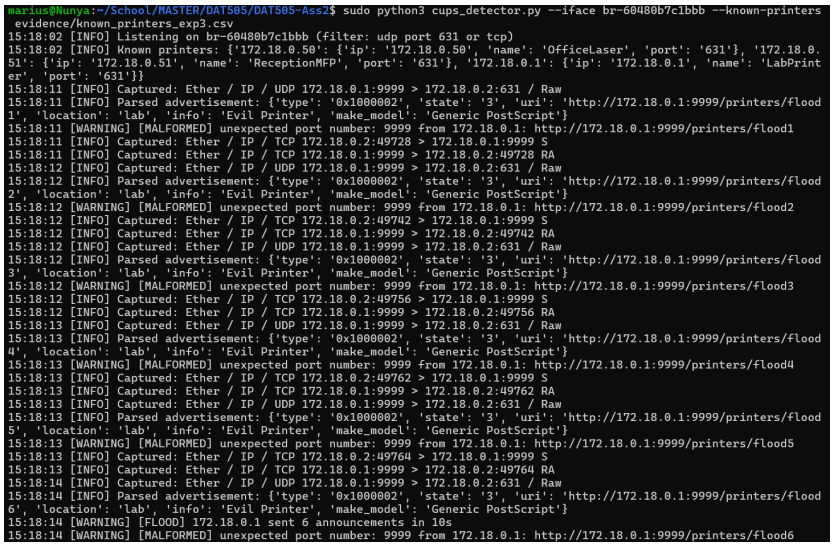


Figure 2: Detector output for the flood variation. Each announcement raises `[MALFORMED]`, and once more than five arrive within the window the detector raises `[FLOOD]`.

3.4 Experiment 4: Detection evaluation

Successful detections (true positives). Advertisements from an unknown address are caught by `[UNAUTHORIZED AD]`. The coerced IPP callback is caught by `[SUSPICIOUS IPP]`, malformed fields (abnormal state, non-hex type, non-standard port) by `[MALFORMED]`, and rapid repetition by `[FLOOD]`. Every crafted variation in Experiment 3 raised at least one alert.

False positives. The `port != 631` rule flags *any* printer on a non-standard port, including legitimate ones. The lab's own `ippeveprinter` on `8632` would be reported as `[MALFORMED]`. The `not known IP` rule likewise flags every printer absent from the allowlist. A newly added legitimate printer is therefore a false positive until the list is updated.

Missed attacks (false negatives). `[SUSPICIOUS IPP]` is only raised for connections to printers advertised by a *non-allowlisted* source. In the populated-allowlist runs, such as the `evilprinter` advertisement and variation v3, the victim connected back to the attacker yet no `[SUSPICIOUS IPP]` was logged, because the advertiser's IP was trusted. In `evidence/detector_exp2.log` the only alert is the `[MALFORMED]` at line 5, with no `[SUSPICIOUS IPP]`, and the callback itself is in `pcap_files/exp2_attack.pcap`. Consequently a well-formed advertisement on port `631` from an allowlisted *or spoofed* address (Experiment 1's configuration) produces *no alert at all* despite a genuine callback.

Two further gaps remain. IP allowlisting is defeated by UDP source-address spoofing, and the capture filter (`udp port 631 or tcp`) does not observe the mDNS/DNS-SD path (`udp 5353`), which means an mDNS-based registration would be missed entirely.

Summary. The detector reliably catches noisy or malformed attacks but relies on source-IP trust and a fixed port assumption, both of which a careful attacker can sidestep. Signature checks should be treated as heuristics layered on top of connection-behaviour analysis, not as a boundary.

Artifacts. All packet captures and logs are included under `pcap_files/` and `evidence/`, one capture/log pair per experiment, including the flood run (`exp3_v4_flood.pcap` and `detector_exp3_v4_flood.log`). Each run also has a CSV of its alerts alongside the log (for example `evidence/detector_before.csv`).

4 Defence and Mitigation

The vulnerability class exploited here (CVE-2024-47176) stems from `cups-browsed` binding to `INADDR_ANY:631/udp` and trusting any datagram from any source. Every practical mitigation therefore works by removing *reachability* of that socket.

4.1 Mitigations investigated

- **Restrict UDP port 631.** A host firewall rule drops inbound `udp dport 631`, closing the legacy UDP vector while leaving the daemon running. *(Implemented and tested below.)*
- **Disable unnecessary `cups-browsed` functionality.** Stopping and disabling the service removes the listening socket, or `BrowseRemoteProtocols none` in `/etc/cups/cups-browsed.conf` disables the legacy protocol.
- **Allowlist authorised printers.** `BrowseAllow`/`BrowseOrder` at the configuration layer, mirrored by the `--known-printers` list of the detector.
- **Patching and segmentation.** `cups-browsed` 2.1.0 removes legacy CUPS browsing, fixing the flaw. Segmentation limits which hosts can reach the service.

4.2 Implementation and method

The firewall mitigation was applied inside the victim's own network namespace, which makes it genuinely the victim host's firewall and requires no tooling inside the container:

```
PID=$(sudo docker inspect -f '{{.State.Pid}}' "$VICTIM")
sudo nsenter -t "$PID" -n iptables -I INPUT -p udp --dport 631 -j DROP
```

The identical attack (`cups_attack.py --adv-ip 172.18.0.1 --target 172.18.0.2`) was run before and after applying the rule.

4.3 Results

The firewall does *not* stop the attacker transmitting the advertisement. The detector still observes it on the wire, because packet capture occurs before the kernel's `INPUT` filtering. What the firewall prevents is `cups-browsed` acting on the advertisement, and the harmful callback no longer occurs (Figure 3).

Observation	Before (no firewall)	After (`DROP udp/631`)
Advertisement on the wire	`[UNAUTHORIZED AD]`	`[UNAUTHORIZED AD]` (still)
`cups-browsed` callback	`VICTIM CONNECTED BACK`	none
Detector IPP alert	`[SUSPICIOUS IPP]` ($\times 5$)	none
Attacker tool verdict	HTTP `POST` received	`No callback within 5s`
`iptables` drop counter	n/a	`1 pkt / 125 B`

Table 2: Effect of the UDP-631 drop rule.

After applying the mitigation.

```
[INFO] Sent announcement to 172.18.0.2:631
[INFO] No callback within 5s --- patched / unreachable?

pkts bytes target     prot  ...  destination
   1   125 DROP       udp   ...  udp dpt:631
```

Evidence. The `[SUSPICIOUS IPP]` present at line 8 of `evidence/detector_before.log` is absent from `evidence/detector_after.log`, whose only alerts are at lines 5 and 6. The captures are `pcap_files/before_mitigation.pcap` and `pcap_files/after_mitigation.pcap`, and the firewall counter is at line 1 of `evidence/iptables_drop_counter.txt`.

4.4 Limitations

Blocking UDP 631 defeats the legacy UDP vector specifically. It does *not* close the LAN mDNS/DNS-SD path. IP-based allowlisting is a hardening measure rather than a boundary, since UDP source addresses can be spoofed. As the evidence shows, the firewall does not prevent an attacker from *sending* advertisements, only from having them acted upon, and it should therefore be combined with monitoring. The only complete fix is patching to a version that removes legacy CUPS browsing. The firewall, service disablement, allowlisting, and segmentation are defence-in-depth layers.

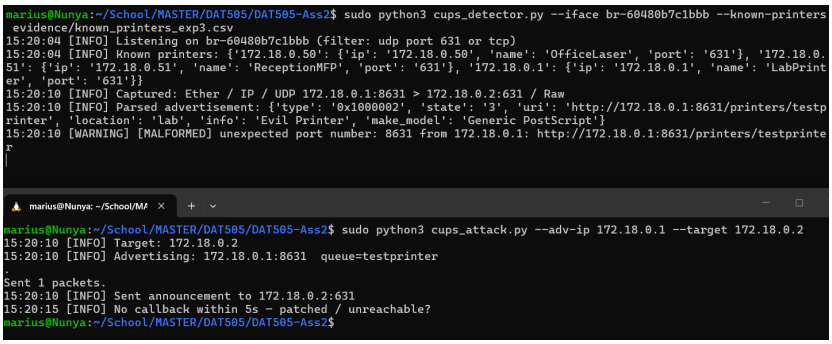


Figure 3: After the UDP-631 drop rule is applied, the attack tool reports `No callback within 5s` and the detector records the advertisement but no `[SUSPICIOUS IPP]`, confirming the victim no longer connects back.

5 Ethical Considerations

As with everything in this course, there are ethical considerations to weigh. The aim of this work is to understand how these attacks function, why they are possible, and why deprecated systems are deprecated. Knowing how an attacker thinks and what they can do lets us mitigate the damage when we cannot remove it entirely. As shown earlier, blocking port 631 removes the callback that the CUPS exploit chain relies on.

The assignment was carried out entirely locally. Vulhub is an open-source collection of reproducible CVE environments for learning how real systems behave when attacked, which shows what to look for and how to mitigate these attack surfaces. The Vulhub image was deployed on the local Docker client, and the attack tool was used only against that container.

The attack tool itself does no real damage. It only signals that the vulnerable host has connected back, and then it stops. That said, building such a tool could serve as a stepping stone, and a real attacker could take it and develop it into a genuine attack.